

PROJETO CHAMADA SMART

DOCUMENTO DE ARQUITETURA E REQUISITOS (AEP 1º BIMESTRE)

Instituição: Unicesumar (Maringá - PR)

1. DESCOBERTA E CONCEPÇÃO (O PROBLEMA E A ODS)

O professor Ricardo, docente de Biologia, enfrenta diariamente o desgaste de perder cerca de 10 minutos de sua aula apenas para realizar a chamada em voz alta, além de carregar diários de papel pesados que correm o risco de molhar ou rasgar. Aos finais de semana, sua carga de trabalho é multiplicada pela correção manual de pilhas de provas. Somado a isso, a infraestrutura de internet das escolas em que leciona é frequentemente instável, impossibilitando o uso de ferramentas em nuvem convencionais durante a aula, o que gera frustração e perda de tempo útil de ensino.

Este projeto alinha-se diretamente à **ODS 4 da ONU (Educação de Qualidade)**. Ao automatizar a gestão de frequência e a aplicação de quizzes, o tempo precioso recuperado em sala de aula pode ser redirecionado para abordagens pedagógicas mais ativas. A redução da sobrecarga burocrática dos professores reflete diretamente na melhoria da qualidade do ensino, no acompanhamento mais assertivo do desempenho dos estudantes e na mitigação da evasão escolar por desengajamento.

Resumo do Discovery (Matriz CSD e Jornada)

Durante a fase de descoberta, validou-se por meio da Jornada do Usuário que a principal barreira tecnológica era a oscilação de conectividade na sala de aula. A Matriz CSD (Certezas, Suposições e Dúvidas) revelou uma premissa técnica inegociável: o sistema precisa ter uma arquitetura **Offline-first**. O aplicativo deve permitir o registro local e seguro das presenças e avaliações, sincronizando os dados com o servidor apenas quando o dispositivo do professor detectar uma conexão estável à internet.

2. LISTA DE REQUISITOS FUNCIONAIS (ESCOPO)

- **RF01** - O sistema deve permitir a realização de chamadas rápidas com registro de presença em lote de forma offline, preservando os dados no armazenamento local do dispositivo.
- **RF02** - O sistema deve permitir o registro de "presença parcial" para alunos atrasados, contabilizando frações exatas de presença e ausência com base no horário de registro.
- **RF03** - O sistema deve permitir a sincronização automática dos dados de frequência e quizzes (modo offline-first) com o banco de dados em nuvem assim que uma conexão de rede estável for detectada.
- **RF04** - O sistema deve permitir a criação, execução e correção automatizada de um motor de quizzes interativos offline, atribuindo as notas diretamente no diário digital do professor.
- **RF05** - O sistema deve permitir a geração de relatórios consolidados de frequência e desempenho por turma e aluno, possibilitando a exportação segura das informações.

3. O PLANEJAMENTO ÁGIL (CRONOGRAMA / BACKLOG)

Abaixo, apresenta-se o fatiamento vertical das entregas simulando o Roadmap de desenvolvimento do 2º Bimestre.

Sprint / Data	Épico	User Story	Responsável
Sprint 1 05/09 - 19/09	Gestão de Turmas	COMO UM Professor de Biologia EU QUERO cadastrar e visualizar minhas turmas PARA QUE eu possa organizar minhas chamadas diárias.	João
Sprint 2 20/09 - 04/10	Motor Offline	COMO UM Professor EU QUERO registrar presenças e faltas parciais localmente PARA QUE eu não dependa da internet instável da escola.	Maria
Sprint 3 05/10 - 19/10	Sync em Nuvem	COMO UM Professor EU QUERO que o app sincronize os dados automaticamente ao detectar rede PARA QUE eu não precise fazer backup manual.	Pedro
Sprint 4 20/10 - 03/11	Motor de Quizzes	COMO UM Professor EU QUERO aplicar testes com correção automática PARA QUE eu evite gastar meu final de semana corrigindo provas físicas.	João e Maria

4. JUSTIFICATIVA TÉCNICA E ARQUITETURAL

Para garantir um ecossistema robusto e tolerante a falhas, o Back-end da solução no 2º Bimestre será construído utilizando **Java com Spring Boot**. A escolha pela linguagem Java fundamenta-se em sua tipagem forte e sólida implementação dos pilares da **Orientação a Objetos**. O domínio do problema exige regras de negócio rigorosas (cálculos de falta parcial, correção polimórfica de avaliações, perfis de acesso). O uso adequado de abstração, encapsulamento, herança e polimorfismo garantirá um código testável, coeso e manutenível, em estrita aderência às exigências da disciplina de Engenharia de Software.

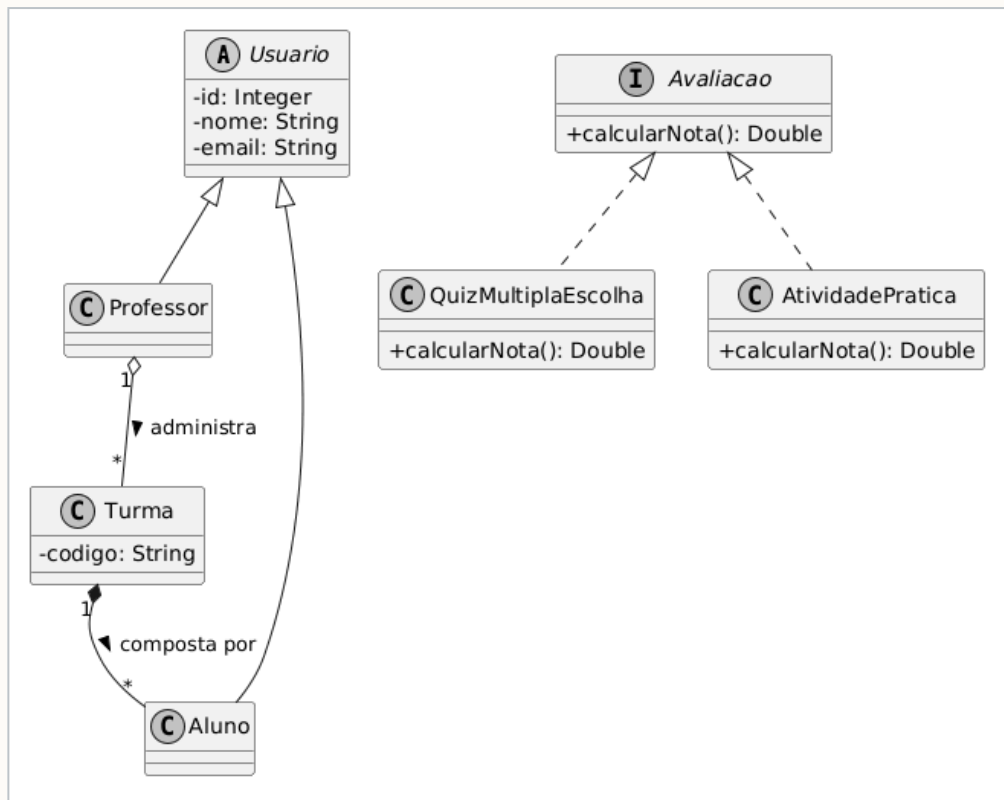
No tange à persistência de dados e estrutura de comunicação, o projeto adotará o banco de dados relacional **PostgreSQL** orquestrado sob o padrão arquitetural de **API REST** ou **MVC**. O PostgreSQL assegura a integridade referencial necessária para mapear as complexas cardinalidades entre as entidades Professor, Turmas e Alunos. A abordagem via API REST permite o desacoplamento estratégico do Front-end (desenvolvido para operar offline) em relação ao servidor. Dessa forma, o aplicativo pode empacotar requisições e enviá-las assincronamente ao servidor apenas quando houver conectividade, contornando a alta latência da infraestrutura escolar sem comprometer a confiabilidade dos registros.

5. ESTRUTURAÇÃO DO GITHUB E DIAGRAMAS

5.1 Link do Repositório

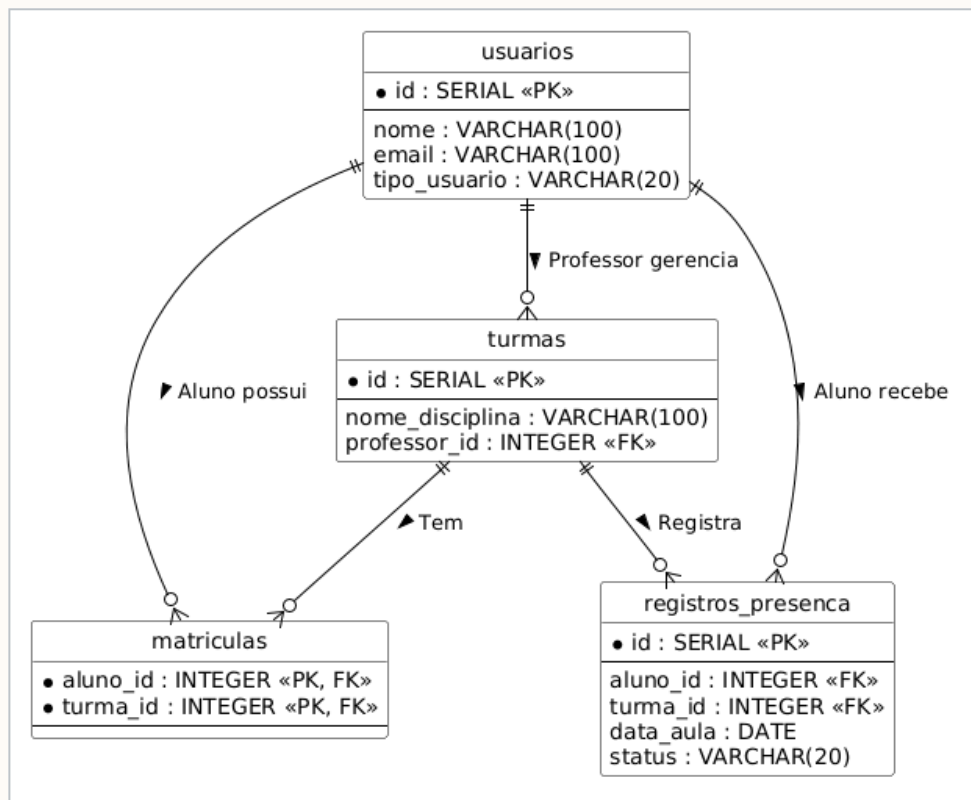
- **Repositório Público GitHub:** <https://github.com/equipe-alfa-aep/chamada-smart-2026>

5.2 Diagrama de Classes (UML)



- **Herança:** Criação de uma superclasse genérica e abstrata chamada **Usuario** (com atributos comuns como `id`, `nome` e `email`). Desta classe, derivam as subclasses específicas **Professor** e **Aluno**, reaproveitando código e isolando responsabilidades.
- **Composição 1:N:** A classe **Professor** detém uma lista agregada da classe **Turma** (um professor administra múltiplas turmas). Por sua vez, a classe **Turma** é composta por múltiplas instâncias de **Aluno**.
- **Polimorfismo:** Implementação de uma interface ou superclasse **Avaliacao** contendo o método de assinatura `calcularNota()`. Este método é sobrescrito (`@Override`) de formas distintas pelas classes filhas **QuizMultiplaEscolha** e **AtividadePratica**, alterando dinamicamente a lógica de correção.

5.3 Diagrama de Banco de Dados (DER)



- **Tabela usuarios:** Centraliza credenciais e dados pessoais, com uma coluna discriminadora (tipo_usuario) para distinguir professores de alunos.
- **Tabela turmas:** Registra os metadados das disciplinas, possuindo a chave estrangeira professor_id referenciando quem as gerencia.
- **Tabela matriculas (Associativa):** Resolve o relacionamento N:M entre estudantes e disciplinas, armazenando simultaneamente aluno_id e turma_id.
- **Tabela registros_presenca:** Armazena o detalhamento de cada chamada, contendo as chaves estrangeiras aluno_id e turma_id, além das colunas de rastreo como data_aula e status (Presente, Parcial, Ausente).